

Année Universitaire 2025/2026

# Simulateur du Problème des Lecteurs-Écrivains

Programmation Concurrente - Mini Projet

Membres du groupe :

---

EL Mazyani Mohamed Ilyas

---

Marouane mellakh

---

IDrissa abou nana-aichatou

---

Essadki Naoufal

---

Chaimaa Mousaly

14 mai 2026

# Table des matières

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Contexte et objectifs du travail</b>                            | <b>1</b>  |
| 1.1      | Contexte . . . . .                                                 | 1         |
| 1.2      | Objectifs du travail . . . . .                                     | 1         |
| <b>2</b> | <b>Structure du projet</b>                                         | <b>1</b>  |
| 2.1      | Organisation des fichiers . . . . .                                | 1         |
| 2.2      | Interaction entre les classes . . . . .                            | 1         |
| 2.3      | Description des classes . . . . .                                  | 2         |
| <b>3</b> | <b>Explication détaillée du code source</b>                        | <b>2</b>  |
| 3.1      | Classe Main - Point d'entrée . . . . .                             | 2         |
| 3.2      | Classe RessourcePartagee - Le cœur de la synchronisation . . . . . | 3         |
| 3.3      | Classe Simulateur - Orchestrateur . . . . .                        | 4         |
| 3.4      | Classe Lecteur - Le thread lecteur . . . . .                       | 6         |
| 3.5      | Classe Ecrivain - Le thread écrivain . . . . .                     | 7         |
| <b>4</b> | <b>Implémentation de l'interface graphique</b>                     | <b>8</b>  |
| 4.1      | Interface au démarrage . . . . .                                   | 8         |
| 4.2      | Déroulement d'une simulation . . . . .                             | 9         |
| 4.3      | Fin de simulation et statistiques . . . . .                        | 9         |
| <b>5</b> | <b>Compilation et exécution</b>                                    | <b>10</b> |
| <b>6</b> | <b>Conclusion</b>                                                  | <b>10</b> |
| 6.1      | Résumé du travail réalisé . . . . .                                | 10        |
| 6.2      | Difficultés rencontrées . . . . .                                  | 10        |
| 6.3      | Conclusion générale . . . . .                                      | 10        |

# 1 Contexte et objectifs du travail

## 1.1 Contexte

Le problème des lecteurs-écrivains est un problème classique de la programmation concurrente. Il met en évidence les difficultés de synchronisation lorsqu'un ensemble de processus (lecteurs) souhaitent lire une ressource partagée tandis que d'autres (écrivains) souhaitent la modifier simultanément.

## 1.2 Objectifs du travail

Les objectifs fixés par le cahier des charges sont :

1. **Modélisation** : Implémenter des threads représentant les lecteurs et les écrivains.
2. **Synchronisation** : Créer des mécanismes permettant aux lecteurs de lire simultanément et aux écrivains d'écrire en exclusivité.
3. **Journalisation** : Enregistrer et afficher toutes les actions pour faciliter l'analyse.
4. **Visualisation** : Développer une interface graphique pour visualiser en temps réel les interactions.
5. **Prévention de la famine** : Implémenter une stratégie FIFO (premier arrivé, premier servi).

# 2 Structure du projet

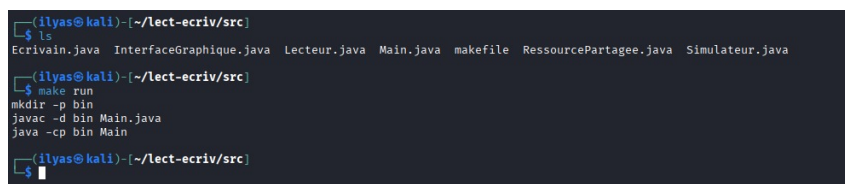
## 2.1 Organisation des fichiers

Le projet est organisé de la manière suivante :

lect-ecriv/

|                         |                                       |
|-------------------------|---------------------------------------|
| Main.java               | # Point d'entrée du programme         |
| InterfaceGraphique.java | # Interface graphique (Swing)         |
| Simulateur.java         | # Orchestrateur de la simulation      |
| RessourcePartagee.java  | # Ressource partagée avec verrou FIFO |
| Lecteur.java            | # Comportement d'un lecteur           |
| Ecrivain.java           | # Comportement d'un écrivain          |
| makefile                | # Script de compilation               |

## 2.2 Interaction entre les classes



```
(ilyas@kali)~/lect-ecriv/src
$ ls
Ecrivain.java  InterfaceGraphique.java  Lecteur.java  Main.java  makefile  RessourcePartagee.java  Simulateur.java
(ilyas@kali)~/lect-ecriv/src
$ make run
mkdir -p bin
javac -d bin Main.java
java -cp bin Main
(ilyas@kali)~/lect-ecriv/src
$
```

FIGURE 1 – Affichage de la structure des fichiers dans le terminal

**Explication :** Cette capture montre la liste des fichiers sources du projet. La commande `make run` permet de compiler et d'exécuter le programme.

## 2.3 Description des classes

| Classe             | Rôle                                                 |
|--------------------|------------------------------------------------------|
| Main               | Point d'entrée, lance l'interface graphique          |
| InterfaceGraphique | Fenêtre Swing, contient les contrôles et l'affichage |
| Simulateur         | Crée, démarre et arrête les threads                  |
| RessourcePartagee  | Ressource protégée par un verrou FIFO                |
| Lecteur            | Thread qui lit la ressource                          |
| Ecrivain           | Thread qui écrit dans la ressource                   |

## 3 Explication détaillée du code source

### 3.1 Classe Main - Point d'entrée

```
import javax.swing.SwingUtilities;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                InterfaceGraphique gui = new InterfaceGraphique();
                gui.setVisible(true);
            }
        });
    }
}
```

FIGURE 2 – Code de la classe Main

**Explication ligne par ligne :**

- `SwingUtilities.invokeLater()` : Cette méthode garantit que l'interface graphique est créée dans le thread dédié (Event Dispatch Thread). C'est une obligation en Swing pour éviter les problèmes de concurrence.
- `new InterfaceGraphique()` : Crée une nouvelle instance de la fenêtre principale.
- `setVisible(true)` : Rend la fenêtre visible à l'écran.

### 3.2 Classe RessourcePartagee - Le cœur de la synchronisation

```
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class RessourcePartagee {

    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock(fair: true);
    private String donnee = "Initial";
    private InterfaceGraphique gui;
    private static int compteurFile = 0;

    public RessourcePartagee(InterfaceGraphique gui) {
        this.gui = gui;
        gui.ajouterMessage("Ressource initialisee: " + donnee);
    }

    public void lire(int id) {
        int position;
        synchronized(this) {
            compteurFile++;
            position = compteurFile;
        }
        gui.ajouterMessage("[FILE] Lecteur " + id + " entre en file (position " + position + ")");

        rwLock.readLock().lock();
        gui.ajouterMessage("[LECTURE] Lecteur " + id + " lit: " + donnee);
        rwLock.readLock().unlock();

        gui.ajouterMessage("[SORTIE] Lecteur " + id + " sort de la file");

        try { Thread.sleep(100); } catch (Exception e) {}
    }

    public void ecrire(int id, String valeur) {
        int position;
        synchronized(this) {
            compteurFile++;
            position = compteurFile;
        }
        gui.ajouterMessage("[FILE] Ecrivain " + id + " entre en file (position " + position + ")");

        rwLock.writeLock().lock();
        gui.ajouterMessage("[ECRITURE] Ecrivain " + id + " ecrit: " + valeur);
        donnee = valeur;
        gui.majValeur(valeur);
        rwLock.writeLock().unlock();

        gui.ajouterMessage("[SORTIE] Ecrivain " + id + " sort de la file");

        try { Thread.sleep(300); } catch (Exception e) {}
    }

    public String getDonnee() {
        return donnee;
    }
}
```

FIGURE 3 – Code de la classe RessourcePartagee

#### Explication détaillée :

- `ReentrantReadWriteLock(true)` : Crée un verrou avec mode équitable. Le paramètre `true` garantit l'ordre FIFO (premier arrivé, premier servi). C'est cette ligne qui empêche la famine.
- `readLock().lock()` : Verrouillage en lecture. Plusieurs lecteurs peuvent prendre ce verrou simultanément. C'est ce qui permet aux lecteurs de lire en parallèle.
- `writeLock().lock()` : Verrouillage en écriture. Un seul thread peut prendre ce verrou à la fois. Pendant ce temps, aucun lecteur ne peut lire.
- `synchronized(this)` : Bloc synchronisé qui protège l'incrémentation du compteur de file. Cela garantit que chaque thread reçoit un numéro unique.
- Les messages `[FILE]`, `[LECTURE]`, `[ECRITURE]`, `[SORTIE]` permettent de visualiser l'ordre d'exécution dans l'interface.

### 3.3 Classe Simulateur - Orchestrateur

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Simulateur {
5
6      private int nbLecteurs;
7      private int nbEcrivains;
8      private int dureeSimulationMs;
9
10     private RessourcePartagee ressource;
11     private List<Thread> threads;
12     private boolean fini;
13     private InterfaceGraphique gui;
14
15     public Simulateur(InterfaceGraphique gui, int nbLecteurs, int nbEcrivains, int dureeSimulationMs) {
16         this.gui = gui;
17         this.nbLecteurs = nbLecteurs;
18         this.nbEcrivains = nbEcrivains;
19         this.dureeSimulationMs = dureeSimulationMs;
20         this.ressource = new RessourcePartagee(gui);
21         this.threads = new ArrayList<>();
22         this.fini = false;
23     }
24
25     public void demarrer() {
26         for (int i = 1; i <= nbLecteurs; i++) {
27             Thread lecteur = new Thread(new Lecteur(i, ressource, this, gui));
28             lecteur.start();
29             threads.add(lecteur);
30             gui.ajouterMessage("Lecteur " + i + " démarre");
31             try { Thread.sleep(500); } catch (Exception e) {}
32         }
33
34         for (int i = 1; i <= nbEcrivains; i++) {
35             Thread ecrivain = new Thread(new Ecrivain(i, ressource, this, gui));
36             ecrivain.start();
37             threads.add(ecrivain);
38             gui.ajouterMessage("Ecrivain " + i + " démarre");
39             try { Thread.sleep(500); } catch (Exception e) {}
40         }
41
42         gui.ajouterMessage(message: "");
43         gui.ajouterMessage(">>> SIMULATION EN COURS (" + (dureeSimulationMs/1000) + " secondes) <<<");
44
45         try {
46             Thread.sleep(dureeSimulationMs);
47         } catch (Exception e) {}
48
49         arreter();
50     }
51
52     public void arreter() {
53         gui.ajouterMessage(message: "");
54         gui.ajouterMessage(">>> ARRÊT DE LA SIMULATION <<<");
55     }
56

```

FIGURE 4 – Code de la classe Simulateur - Démarrage

#### Explication du démarrage :

- Le constructeur initialise les paramètres : nombre de lecteurs, nombre d'écrivains, durée de la simulation.
- La méthode `demarrer()` crée et lance tous les threads :
  - `new Thread(new Lecteur(...))` : Crée un nouveau thread pour un lecteur.
  - `start()` : Démarre le thread (la méthode `run()` s'exécute alors en parallèle).
  - `threads.add()` : Stocke le thread dans une liste pour pouvoir l'arrêter plus tard.
  - `Thread.sleep(500)` : Pause de 500ms entre chaque démarrage pour mieux visualiser l'ordre d'arrivée.
- `Thread.sleep(dureeSimulationMs)` : Attend la durée configurée avant d'arrêter la simulation.

```
public void arreter() {
    gui.ajouterMessage(message: "");
    gui.ajouterMessage(message: ">>> ARRET DE LA SIMULATION <<<");

    fini = true;

    for (Thread t : threads) {
        t.interrupt();
    }

    for (Thread t : threads) {
        try {
            t.join(millis: 1000);
        } catch (Exception e) {}
    }

    gui.ajouterMessage(message: "");
    gui.ajouterMessage(message: "=== STATISTIQUES FINALES ===");
    gui.ajouterMessage("Lectures: " + Lecteur.getCompteurLectures());
    gui.ajouterMessage("Ecritures: " + Ecrivain.getCompteurEcritures());
    gui.ajouterMessage("Valeur finale: " + ressource.getDonnee());

    gui.majLectures(Lecteur.getCompteurLectures());
    gui.majEcritures(Ecrivain.getCompteurEcritures());
}

public boolean estFini() {
    return fini;
}
```

FIGURE 5 – Code de la classe Simulateur - Arrêt

**Explication de l'arrêt :**

- `fini = true` : Drapeau signalant la fin de la simulation. Les threads vérifient cette variable dans leur boucle.
- `t.interrupt()` : Demande poliment à chaque thread de s'arrêter. Cela réveille les threads qui sont en train de dormir.
- `t.join(1000)` : Attend que le thread se termine (maximum 1 seconde). Cela garantit que tous les threads sont bien arrêtés avant d'afficher les statistiques.
- `getCompteurLectures()` et `getCompteurEcritures()` : Récupèrent les statistiques finales.

### 3.4 Classe Lecteur - Le thread lecteur

```
import java.util.Random;

public class Lecteur implements Runnable {
    private int id;
    private RessourcePartagee ressource;
    private Simulateur simulateur;
    private InterfaceGraphique gui;
    private static int compteur = 0;

    public Lecteur(int id, RessourcePartagee ressource, Simulateur simulateur, InterfaceGraphique gui) {
        this.id = id;
        this.ressource = ressource;
        this.simulateur = simulateur;
        this.gui = gui;
    }

    public static synchronized int getCompteurLectures() {
        return compteur;
    }

    private static synchronized void incrementerCompteur() {
        compteur++;
    }

    public void run() {
        Random rand = new Random();

        while (!simulateur.estFini() && !Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(1500 + rand.nextInt(bound: 2000));
            } catch (Exception e) {
                break;
            }

            ressource.lire(id);
            incrementerCompteur();
            gui.majLectures(compteur);
        }

        gui.ajouterMessage("Lecteur " + id + " arrete");
    }
}
```

FIGURE 6 – Code de la classe Lecteur

#### Explication détaillée :

- `implements Runnable` : Cette interface permet d'exécuter ce code dans un thread. La méthode `run()` contient le code à exécuter.
- `while (!simulateur.estFini() && !Thread.currentThread().isInterrupted())` : La boucle continue tant que la simulation n'est pas finie ET que le thread n'a pas reçu d'ordre d'arrêt.
- `Thread.sleep(1500 + rand.nextInt(2000))` : Pause aléatoire entre 1500ms et 3500ms. Cela simule le temps de réflexion entre deux lectures.
- `ressource.lire(id)` : Appelle la méthode de lecture synchronisée. C'est ici que le thread prend le verrou de lecture.
- `incrementerCompteur()` : Incrémente le compteur total de lectures (partagé entre tous les lecteurs).
- `synchronized` sur les méthodes statiques : Protège l'accès au compteur partagé. Cela évite que deux lecteurs modifient le compteur en même temps.



### 3.5 Classe Ecrivain - Le thread écrivain

```
public class Ecrivain implements Runnable {
    private int id;
    private RessourcePartagee ressource;
    private Simulateur simulateur;
    private InterfaceGraphique gui;
    private static int compteurEcritures = 0;
    private static int compteurModifs = 0;

    public Ecrivain(int id, RessourcePartagee ressource, Simulateur simulateur, InterfaceGraphique gui) {
        this.id = id;
        this.ressource = ressource;
        this.simulateur = simulateur;
        this.gui = gui;
    }

    public static synchronized int getCompteurEcritures() {
        return compteurEcritures;
    }

    private static synchronized String getNouvelleValeur(int id) {
        compteurModifs++;
        return "Donnee-" + compteurModifs + "-par-" + id;
    }

    private static synchronized void incrementerCompteurEcritures() {
        compteurEcritures++;
    }

    public void run() {
        Random rand = new Random();

        while (!simulateur.estFini() && !Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(2000 + rand.nextInt(bound: 2500));
            } catch (Exception e) {
                break;
            }

            String valeur = getNouvelleValeur(id);
            ressource.ecrire(id, valeur);
            incrementerCompteurEcritures();
            gui.majEcritures(compteurEcritures);
        }

        gui.ajouterMessage("Ecrivain " + id + " arrete");
    }
}
```

FIGURE 7 – Code de la classe Ecrivain

#### Explication détaillée :

- Pause aléatoire plus longue : `2000 + rand.nextInt(2500)` (2000ms à 4500ms). Les écrivains sont moins fréquents que les lecteurs.
- `getNouvelleValeur()` : Méthode synchronisée qui :
  - Incrémente le compteur de modifications
  - Retourne une chaîne unique au format `Donnee-X-par-id`
- `ressource.ecrire(id, valeur)` : Appelle la méthode d'écriture synchronisée. C'est ici que le thread prend le verrou d'écriture (exclusif).
- `incrementerCompteurEcritures()` : Incrémente le compteur total d'écritures.

## 4 Implémentation de l'interface graphique

### 4.1 Interface au démarrage

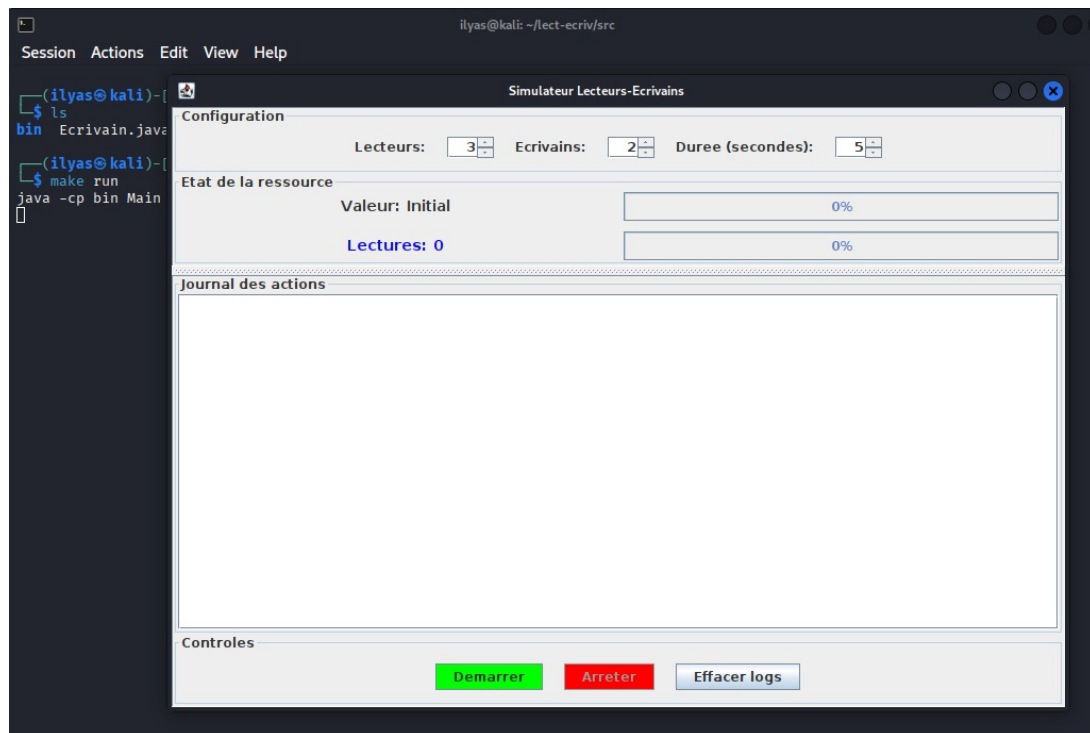


FIGURE 8 – Interface graphique au démarrage

#### Description des composants :

- **Zone Configuration** : Permet de modifier :
  - Le nombre de lecteurs (1 à 10)
  - Le nombre d'écrivains (1 à 10)
  - La durée de la simulation (1 à 30 secondes)
- **Zone Etat de la ressource** : Affiche en temps réel :
  - La valeur actuelle de la ressource
  - Le nombre total de lectures
  - Le nombre total d'écritures
- **Zone Journal des actions** : Affiche toutes les actions avec horodatage.
- **Boutons de contrôle** :
  - Démarrer : Lance la simulation
  - Arrêter : Interrompt la simulation
  - Effacer logs : Vide le journal

## 4.2 Déroulement d'une simulation

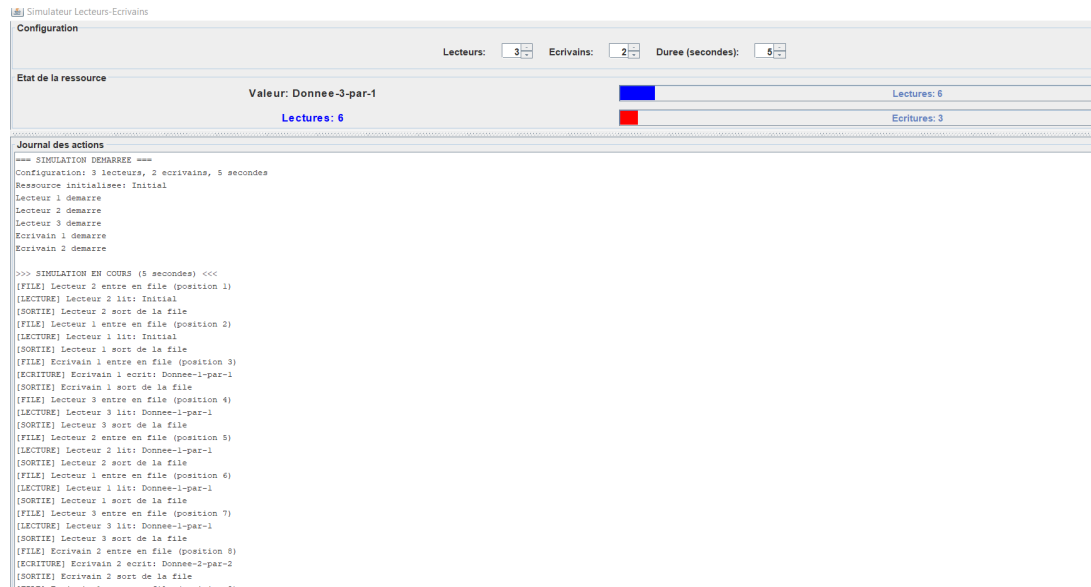


FIGURE 9 – Déroulement de la simulation - Journal des actions

### Explication des messages :

- [FILE] Lecteur X entre en file (position N) : Le lecteur demande l'accès et prend sa place dans la file FIFO.
- [LECTURE] Lecteur X lit : Valeur : Le lecteur a obtenu le verrou et lit la ressource.
- [SORTIE] Lecteur X sort de la file : Le lecteur a terminé et libère le verrou.
- [ECRITURE] Ecrivain X écrit : Nouvelle valeur : L'écrivain modifie la ressource.

## 4.3 Fin de simulation et statistiques

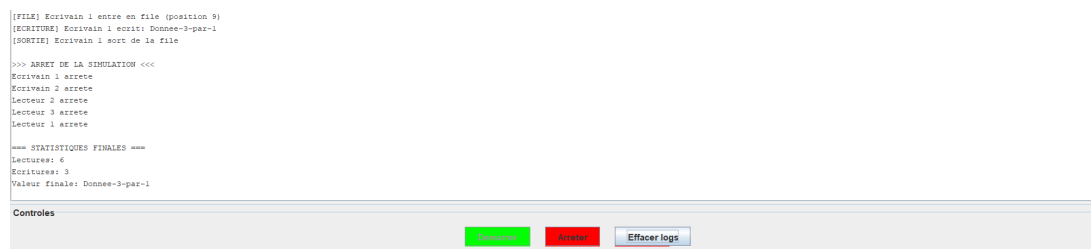
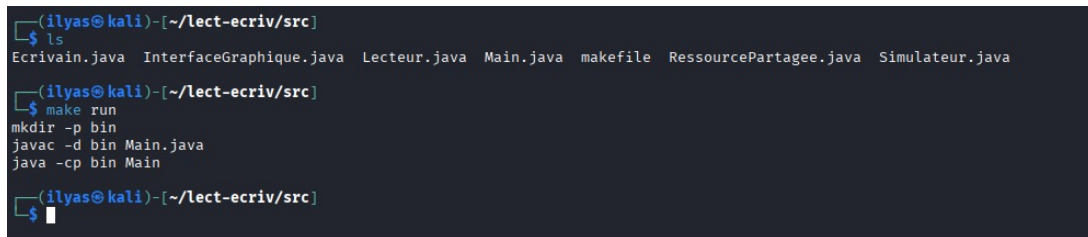


FIGURE 10 – Fin de simulation et statistiques finales

### Explication :

- >> ARRET DE LA SIMULATION << : Signal de fin envoyé par le simulateur.
- Lecteur X arrete / Ecrivain X arrete : Chaque thread confirme son arrêt.
- === STATISTIQUES FINALES === : Résumé de la simulation :
  - Nombre total de lectures effectuées
  - Nombre total d'écritures effectuées
  - Valeur finale de la ressource

## 5 Compilation et exécution



```
(ilyas@kali)-[~/lect-ecriv/src]
$ ls
Ecrivain.java  InterfaceGraphique.java  Lecteur.java  Main.java  makefile  RessourcePartagee.java  Simulateur.java
(ilyas@kali)-[~/lect-ecriv/src]
$ make run
mkdir -p bin
javac -d bin Main.java
java -cp bin Main
(ilyas@kali)-[~/lect-ecriv/src]
$
```

FIGURE 11 – Compilation et exécution avec Makefile

### Explication des commandes :

- `make run` : Commande principale qui compile et exécute le programme.
- `mkdir -p bin` : Crée le dossier `bin` pour les fichiers compilés.
- `javac -d bin Main.java` : Compile la classe `Main` dans le dossier `bin`.
- `java -cp bin Main` : Exécute le programme.

## 6 Conclusion

### 6.1 Résumé du travail réalisé

Nous avons développé un simulateur complet du problème des lecteurs-écrivains répondant à toutes les exigences :

- **Modélisation** : Threads lecteurs et écrivains
- **Synchronisation** : `ReentrantReadWriteLock` avec mode FIFO
- **Journalisation** : Affichage complet des actions
- **Interface graphique** : Swing avec visualisation temps réel
- **Anti-famine** : Stratégie FIFO garantie

### 6.2 Difficultés rencontrées

1. **Synchronisation des compteurs** : Utilisation de `synchronized` pour protéger l'accès aux variables partagées.
2. **Arrêt propre des threads** : Mise en place du drapeau `fini` combiné avec `interrupt()` et `join()`.
3. **Affichage temps réel** : Utilisation de `SwingUtilities.invokeLater()` pour les mises à jour de l'interface.

### 6.3 Conclusion générale

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la programmation concurrente en Java :

- Gestion des threads et de leur synchronisation
- Utilisation des verrous lecture/écriture
- Développement d'une interface graphique temps réel
- Prévention de la famine avec une stratégie FIFO

Le simulateur fonctionne correctement et illustre parfaitement le problème des lecteurs-écrivains ainsi que sa solution.